

Módulo 2 - Entradas/Salidas digitales (GPIO)

Proyecto de identificación RLS/ARX en STM32F405RGT6

Archivos documentados: gpio_driver.h, gpio_driver.c, gpio_if.h y gpio_if.c.



Función del módulo. Configura y manipula pines digitales usados por tres subsistemas: control TFT, dirección del motor y LED RGB.

Pines implementados. PB6, PB7 y PB8 para CS/DC/RST del ILI9341; PA2 y PA3 para la lógica de sentido del motor; PB12, PB13 y PB14 para LED RGB.

Estilo del driver. Usa acceso directo a registros y macros con BSRR para set/reset atómico de salidas.

Uso dentro del proyecto. La aplicación invoca `GPIO_STM32.tft.init()`, `GPIO_STM32.motor.init()` y `GPIO_STM32.rgb.init()` desde `main.c` y otras librerías.

1. Papel del módulo dentro del firmware

Aunque el nombre del archivo sugiere un driver GPIO genérico, en realidad este módulo está orientado a funciones muy concretas del sistema. No intenta abstraer todos los puertos del STM32F405, sino resolver tres necesidades digitales del proyecto: gobernar líneas de control del display TFT, seleccionar el sentido de giro del motor y proporcionar señalización visual con un LED RGB.

Eso lo vuelve muy didáctico para estudiantes, porque muestra una práctica común en firmware bare-metal: crear drivers suficientemente específicos para la aplicación, en lugar de desarrollar una biblioteca enorme y completamente genérica desde el primer día.

2. Relación entre archivos

Archivo	Responsabilidad
gpio_driver.h	Declara macros de control rápido y prototipos de las rutinas de inicialización.
gpio_driver.c	Configura registros RCC, MODER, OTYPER, OSPEEDR y PUPDR para los pines usados.
gpio_if.h	Define estructuras de interfaz separadas para TFT, motor y LED RGB.
gpio_if.c	Construye la instancia constante <code>GPIO_STM32</code> que enlaza macros y funciones del driver con una API uniforme.

Fragmento 1. API de inicialización publicada por el driver

```
void TFT_ctrl_gpio_init(void);
void GPIOB_Init_PB12_13_14_Output(void);
void gpio_pa2_pa3_output_init(void);
```

3. gpio_driver.h: macros de control inmediato

La cabecera define macros para escribir directamente en los pines de salida. Esta técnica evita la sobrecarga de una llamada convencional y resulta útil cuando una acción debe ejecutarse de forma muy frecuente, por ejemplo cambiar DC antes de enviar un comando al TFT o invertir el estado de un LED desde una interrupción.

La idea central es el uso del registro BSRR del GPIO. En STM32, la mitad baja del BSRR pone pines en 1 y la mitad alta los pone en 0. Esto permite modificar salidas sin leer antes el ODR, y por eso se considera una escritura más segura y atómica para set/reset.

Bloque	Macro	Pin	Efecto
TFT	TFT_CS_LOW/HIGH	PB6	Activa o libera el chip select del ILI9341.
TFT	TFT_DC_COMMAND/DATA	PB7	Selecciona si el byte enviado por SPI es comando o dato.
TFT	TFT_RST_LOW/HIGH	PB8	Fuerza reset físico del display y después lo libera.
Motor	left_motor()	PA2/PA3	Pone PA2=1 y PA3=0 para un sentido de giro.
Motor	right_motor()	PA2/PA3	Pone PA2=0 y PA3=1 para el sentido contrario.
RGB	LED_R/G/B_ON/OFF	PB12-14	Enciende o apaga cada canal del LED RGB.
RGB	blink_R/G/B()	PB12-14	Conmuta el bit en ODR mediante XOR.

Fragmento 2. Ejemplos de macros basadas en BSRR y ODR

```
#define TFT_CS_LOW()    (GPIOB->BSRR = 0x00400000U)
#define TFT_CS_HIGH()   (GPIOB->BSRR = 0x00000040U)

#define left_motor()     (GPIOA->BSRR = 0x00080004U)
#define right_motor()    (GPIOA->BSRR = 0x00040008U)

#define LED_R_ON()       (GPIOB->BSRR = 0x00001000U)
#define LED_R_OFF()      (GPIOB->BSRR = 0x10000000U)
#define blink_R()        (GPIOB->ODR ^= 0x00001000U)
```

3.1 Interpretación de las macros del motor

Las macros `left_motor()` y `right_motor()` escriben simultáneamente dos acciones en un solo acceso a BSRR: un bit se coloca en la mitad baja para poner una salida en 1 y otro bit se coloca en la mitad alta para limpiar la otra salida. Por ejemplo, `0x00080004U` significa reset de PA3 y set de PA2. Esta forma es más robusta que hacer dos escrituras separadas porque evita estados intermedios breves.

3.2 Una diferencia importante: set/reset frente a toggle

Las macros de encendido y apagado usan BSRR, mientras que los toggles del LED usan `ODR ^= mask`. Eso funciona bien cuando un solo contexto controla el LED, como ocurre aquí, pero desde el punto de vista didáctico conviene explicar que el toggle sobre ODR es una operación de lectura-modificación-escritura y no es tan atómica como set/reset con BSRR.

4. Configuración de registros en `gpio_driver.c`

Cada rutina de inicialización sigue la secuencia clásica de un pin GPIO en STM32: habilitar reloj del puerto, seleccionar modo en `MODER`, definir tipo de salida en `OTYPER`, escoger velocidad en `OSPEEDR` y configurar resistencias internas con `PUPDR`. La diferencia entre funciones está en qué pines se tocan y cuál es el estado inicial deseado.

Función	Pines	Configuración principal	Estado inicial
TFT_ctrl_gpio_init()	PB6, PB7, PB8	Output, push-pull, alta velocidad, sin pull-up/down.	CS=1, DC=1, RST=1.
GPIOB_Init_PB12_13_14_Output()	PB12, PB13, PB14	Output, push-pull, velocidad media, sin pull-up/down.	No fuerza encendido ni apagado explícito.
gpio_pa2_pa3_output_init()	PA2, PA3	Output, push-pull, velocidad media, sin pull-up/down.	No fija nivel lógico inicial del puente/H-bridge.

Fragmento 3. Inicialización de las líneas de control del TFT

```
void TFT_ctrl_gpio_init(void)
{
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOBEN;

    GPIOB->MODER  &= ~0x0003F000U;
    GPIOB->MODER  |= 0x00015000U;
    GPIOB->OTYPER  &= ~0x000001C0U;
    GPIOB->OSPEEDR |= 0x0003F000U;
    GPIOB->PUPDR  &= ~0x0003F000U;

    GPIOB->BSRR = (1U << 6);
    GPIOB->BSRR = (1U << 7);
    GPIOB->BSRR = (1U << 8);
}
```

4.1 ¿Por qué el TFT se inicializa con salidas en alto?

La secuencia elegida es coherente con la electrónica del display. CS en 1 deja al periférico deseleccionado, RST en 1 evita mantenerlo en reset permanente y DC en 1 deja la línea en modo dato por defecto. Más adelante la librería del ILI9341 baja RST de forma controlada cuando necesita reinicializar el controlador.

4.2 El detalle de la lectura dummy del reloj

En la función del LED RGB aparece (void)RCC->AHB1ENR; después de habilitar el reloj. Esa lectura dummy es una práctica defensiva usada para asegurar que la escritura al reloj del puerto ya surtió efecto antes de tocar los registros del GPIO. En este microcontrolador muchas veces no es indispensable, pero es una costumbre razonable en bare-metal.

Fragmento 4. Parte central de la inicialización del LED RGB

```
void GPIOB_Init_PB12_13_14_Output(void)
{
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOBEN;
    (void)RCC->AHB1ENR;

    GPIOB->MODER &= ~( (3U << (12U*2U)) |
                      (3U << (13U*2U)) |
                      (3U << (14U*2U)) );
    GPIOB->MODER |=  ( (1U << (12U*2U)) |
                      (1U << (13U*2U)) |
                      (1U << (14U*2U)) );
}
```

5. Capa de interfaz: gpio_if

Este proyecto no deja toda la aplicación acoplada al archivo gpio_driver.c. En su lugar crea una estructura compuesta por tres subinterfases: una para TFT, otra para dirección del motor y otra para el LED RGB. Cada subinterfaz reúne punteros a función con firmas homogéneas y se expone mediante una sola constante global: GPIO_STM32.

Fragmento 5. Dos subinterfases definidas en gpio_if.h

```
typedef struct {
    gpio_void_fn_t init;
    gpio_void_fn_t cs_low;
    gpio_void_fn_t cs_high;
    gpio_void_fn_t dc_command;
    gpio_void_fn_t dc_data;
    gpio_void_fn_t rst_low;
    gpio_void_fn_t rst_high;
} gpio_tft_if_t;

typedef struct {
    gpio_void_fn_t init;
    gpio_void_fn_t left;
    gpio_void_fn_t right;
} gpio_motor_if_t;
```

Fragmento 6. Construcción parcial de la instancia GPIO_STM32

```
const gpio_if_t GPIO_STM32 = {
    .tft = {
        .init      = TFT_ctrl_gpio_init,
        .cs_low    = tft_cs_low_impl,
        .cs_high   = tft_cs_high_impl,
        .dc_command = tft_dc_cmd_impl,
        .dc_data    = tft_dc_data_impl,
        .rst_low    = tft_rst_low_impl,
        .rst_high   = tft_rst_high_impl,
    },
    .motor = {
        .init = gpio_pa2_pa3_output_init,
        .left  = motor_left_impl,
        .right = motor_right_impl,
    }
};
```

5.1 ¿Para qué sirven los wrappers del archivo gpio_if.c?

Las macros como `TFT_CS_LOW()` o `LED_R_ON()` no son funciones y por tanto no pueden asignarse directamente a un puntero a función de forma portable. Por eso el archivo `gpio_if.c` define pequeños wrappers `static` que ejecutan la macro y sí pueden exponerse como callbacks dentro de la interfaz.

6. Secuencia de uso real en la aplicación

El valor didáctico del módulo aumenta cuando se observa dónde se usa. En este firmware los GPIO no aparecen como un tema aislado; participan de forma visible en el arranque de la pantalla, en el control del motor y en la instrumentación del lazo temporal.

Fragmento 7. Uso del GPIO desde main.c

```
int main(void)
{
    SYSCLK_STM32.init();
    ini_pantalla();
    quad_STM32.init();
    PWM_STM32.motor.init(PWM_TARGET_HZ);
    usart_STM32.init_115200();
    GPIO_STM32.motor.init();
    GPIO_STM32.rgb.init();

    while (1)
    {
        GPIO_STM32.motor.left();
        PWM_STM32.motor.set_duty_permille(g_motor.d);
    }
}
```

Fragmento 8. Ejemplo de cooperación entre GPIO, reloj y SPI en la librería ILI9341

```

void reset_TFT(void)
{
    GPIO_STM32.tft.rst_low();
    SYSCLK_STM32.delay_ms(12);
    GPIO_STM32.tft.rst_high();
    SYSCLK_STM32.delay_ms(122);
}

void ili_send_CMD(uint8_t cmd)
{
    GPIO_STM32.tft.dc_command();
    SPI1_STM32.send_byte(cmd);
}

```

Uso	Línea GPIO implicada	Objetivo
Arranque del TFT	PB6, PB7, PB8	Habilitar reset, comando/dato y selección del display sobre SPI.
Sentido de giro	PA2, PA3	Definir la polaridad lógica que alimenta la etapa de potencia del motor.
Indicador temporal	PB12	Parpadear LED rojo dentro de TIM4_IRQHandler() para observar actividad del temporizador.

7. Observaciones didácticas y mejoras posibles

- Usar BSRR para set/reset es una buena práctica de bajo nivel y conviene enseñarla desde temprano.
- Este módulo mezcla tres dominios funcionales distintos. Para un proyecto académico está bien; en un firmware más grande quizá convendría separarlos en archivos `tft_gpio`, `motor_dir_gpio` y `rgb_gpio`.
- La inicialización del motor no fija un estado neutro explícito al final. Si el hardware de potencia lo requiere, podría añadirse una escritura inicial que deje PA2 y PA3 en un estado seguro.
- La inicialización del LED RGB tampoco impone un apagado explícito; si el reset previo dejó basura en ODR, podría ser útil limpiar los bits PB12-PB14 al terminar la configuración.
- Los toggles por XOR son prácticos para depuración visual, pero cuando varios contextos escriben la misma salida es preferible usar un mecanismo más controlado.

8. Conclusión del módulo

El bloque GPIO muestra cómo un periférico aparentemente simple se vuelve esencial cuando se integra con el resto del firmware. Gracias a este módulo el sistema puede conversar con el display TFT, definir el sentido lógico del motor y señalar eventos de ejecución. Desde el punto de vista formativo, el alumno aprende tres ideas clave: cómo configurar un puerto GPIO a nivel de registros, por qué BSRR es útil en STM32 y cómo envolver un driver directo con una capa de interfaz más clara para la aplicación.